

A Zero-Knowledge Proof of Possession of a Formal Development

Gadget templates, two-sided clamps, quantitative soundness,
and the meta-meme cycle as a single 28512-state object

Formalised in Lean 4 with Mathlib

Abstract

This paper presents a zero-knowledge proof of possession (ZKPoP) whose object is a formal development: the joint configuration of the five finite objects that the accompanying Lean project counts and proves theorems about. The construction reuses a shelf of Σ -protocol *gadgets* indexed by capacity, maps objects into the shelf by injective encodings, and pins each object between two consecutive gadgets — a *clamp*. Everything stated here is machine-checked: completeness, 2-special soundness with an explicit extractor, and perfect honest-verifier zero knowledge are proved once for every gadget, with no computational hardness assumption, since they are theorems of linear algebra over a prime field.

Three results improve on the earlier shelf. First, the soundness statement is made *quantitative*: against a false statement a committed prover has at most one answerable challenge, hence success probability at most $1/p$. Second, gadgets are shown to *compose*: the clamp of a product of objects is obtained by adding the template sizes, and the size index of a product lies between the sum of the indices and that sum plus one. Third, these are spent on a single run whose object is the project’s own state space, of cardinality $3 \cdot 4 \cdot 8 \cdot 11 \cdot 27 = 28512$; it is clamped between the ternary templates of sizes 9 and 10, the lower one provably too small, and one run in the upper one proves possession of a whole configuration while revealing exactly the clamp and nothing more. The composition bound is attained: the whole costs precisely one trit more than the sum of its parts.

1 Introduction

The project this paper reports on grew out of a slogan — *the message is the medium is the meme is the model is the morphism* — formalised as a four-stage cycle with a cyclic symmetry, and then pushed into the algebra it suggested: the Frobenius automorphism of $\text{GF}(27)$, its orbits, the monic irreducible cubics over $\mathbb{F}_3$, and a complexity lattice in which finite objects are located between known nodes. What is Noetherian about that development is not the phrase but its method: each object is described by what is *invariant* about it under change of presentation.

A zero-knowledge proof of possession is a natural place to put that method to work. A prover who possesses an element x of some finite object X — a stage of the cycle, an orbit, a field element, or a full configuration of all of them — wants to convince a verifier of possession without transmitting x . What the verifier does learn is a size bracket: which gadget the run took place in, hence a two-sided bound on $|X|$. That bracket is an invariant of X : it depends only on cardinality, and is therefore conserved under every re-encoding.

Contributions. Sections 2–4 recall the construction as it stands in the development (the “example” improved on here): one Σ -protocol proved once for a whole shelf of gadgets, encodings, and two-sided clamps that are tight and unique. Section 5 contains the new material: quantitative soundness (Theorem 5.2), composition of gadgets (Theorem 5.3), and the run on

the project itself (Section 5.3). Section 6 discusses what a verifier learns and the Noetherian reading of the clamp. Appendix A lists every formal name and Appendix B the reproduction instructions.

Status of the claims. Every numbered statement below is a theorem in the Lean development, given by name; the whole library builds with no `sorry` and depends only on Lean’s standard axioms (`propext`, `Classical.choice`, `Quot.sound`). Statements that are *not* machine-checked — interpretive remarks, and the file-level counts of Section 6 — are flagged as such.

2 The gadget and its Σ -protocol

Definition 2.1 (`ZKPoP.Gadget`). A *gadget* g consists of a modulus p , dimensions $k = \text{witnessDim}$ and $m = \text{statementDim}$, and a matrix $R \in \mathbb{F}_p^{m \times k}$. Its witness space is $\text{Wit}(g) = \mathbb{F}_p^k$, its statement space $\mathbb{F}_p^m$, and the relation it proves possession for is

$$x = R w, \quad w \in \text{Wit}(g) \text{ secret}, \quad x \text{ public}.$$

Its *capacity* is $\text{cap}(g) = p^k$, which is the cardinality of $\text{Wit}(g)$ (`Gadget.card_wit`).

The protocol is the standard three-move preimage protocol for a linear map:

Prover (holds w with $Rw = x$)	Verifier (holds x)
sample $r \leftarrow \mathbb{F}_p^k$, set $a := Rr$	$\xrightarrow{a}$
	$\xleftarrow{c}$ sample $c \leftarrow \mathbb{F}_p$
set $z := r + cw$	$\xrightarrow{z}$ check $Rz = a + cx$

Three theorems are proved for *every* gadget at once. None of them assumes a hardness assumption; they are facts about the linear algebra.

Theorem 2.2 (Completeness; `Gadget.accepts_prove`). *For all $w, r \in \text{Wit}(g)$ and all $c \in \mathbb{F}_p$, the transcript $(Rr, c, r + cw)$ is accepted against the statement Rw .*

Theorem 2.3 (2-special soundness; `Gadget.extract_spec`). *Let p be prime and let $(a, c_1, z_1), (a, c_2, z_2)$ be accepting transcripts for the same statement x with $c_1 \neq c_2$. Then*

$$w = (c_1 - c_2)^{-1}(z_1 - z_2)$$

satisfies $Rw = x$. Consequently x is a true statement (`Gadget.special_soundness`).

Theorem 2.4 (Perfect honest-verifier zero knowledge; `Gadget.honest_eq_accepting`). *The simulator $\text{simulate}(x, c, z) = (Rz - cx, c, z)$ takes no witness and always produces an accepting transcript (`Gadget.accepts_simulate`). Moreover, for a fixed challenge c , the transcripts an honest prover holding w can produce are exactly the accepting transcripts at c ; the two parametrizations differ by the bijection $r \mapsto r + cw$ (`Gadget.honest_response_bijective`).*

Theorem 2.4 is the strong, set-level form of zero knowledge: it is not merely that a simulator exists, but that the honest prover’s transcript set and the simulator’s coincide, so a transcript carries no information about w beyond the truth of x .

3 The shelf: templates indexed by capacity

Gadgets are not invented per object. The *template* $T(p^k) = \text{template } p \ k$ has witness space $\mathbb{F}_p^k$, the single public statement $w_1 + \dots + w_k$ (relation matrix the row of ones), and capacity p^k . Capacities are strictly increasing in k (`capacity_template_strictMono`), so the shelf has no duplicates and is ordered by size:

$$1, p, p^2, p^3, \dots$$

Over $\mathbb{F}_3$ the small templates are named `trit` (3), `ditrit` (9), `gf27Template` (27), `quadtrit` (81); the ladder of capacities up to size 5 is $[1, 3, 9, 27, 81, 243]$ (`ladder_three_capacities`), and the witness space of `gf27Template` is in bijection with $\text{GF}(27)$ itself (`gf27_equiv_witness`). Every template inherits Theorems 2.2–2.4 unchanged; what varies along the shelf is capacity alone.

4 Encodings and two-sided clamps

Definition 4.1 (`ZKPoP.Encoding`, `ZKPoP.Clamped`). An *encoding* of a finite object X into a gadget g is an injection $X \hookrightarrow \text{Wit}(g)$. A *clamped object* `Clamped` $X \ L \ U$ is a pair of injections $\text{Wit}(L) \hookrightarrow X \hookrightarrow \text{Wit}(U)$.

Theorem 4.2 (Selection by size; `nonempty_encoding_iff`). *For finite X and $p \neq 0$, an encoding $X \hookrightarrow \text{Wit}(g)$ exists if and only if $|X| \leq \text{cap}(g)$. Failure is an obstruction, not a convention (`not_nonempty_encoding`).*

Theorem 4.3 (The clamp is the bracket; `clamp_iff_clamped`). *Clamped $X \ L \ U$ is inhabited if and only if $\text{cap}(L) \leq |X| \leq \text{cap}(U)$. The pair of embeddings and the numeric bracket are the same information.*

Theorem 4.4 (Tightness and uniqueness; `tight_clamp`, `tightClamp_index_unique`). *For $p \geq 2$ and $n \geq 1$ put $j = \text{clampIndex } p \ n = \log_p n$ (integer logarithm). Then $p^j \leq n < p^{j+1}$, and j is the only index with that property. Consecutive capacities differ by exactly the factor p (`capacity_succ_template`).*

Running the protocol on an encoded object preserves all three guarantees: `Clamped.accepts_run`, `Clamped.run_eq_simulated`, `Clamped.soundness_run`. Finally, clamping is identified with the *sandwich* relation of the project’s complexity lattice (`clamp_iff_sandwich`); encodability is that lattice’s `Fits` and failure its `Breaks`.

The runs already on the shelf. Five objects of the development, each with its state count taken from the theorem that proves it:

object	states	lower	upper	index
$\text{Gal}(\text{GF}(27)/\mathbb{F}_3)$	3	<code>trit</code> (3)	<code>ditrit</code> (9)	1
<code>MetaNode</code> (the four-stage cycle)	4	<code>trit</code> (3)	<code>ditrit</code> (9)	1
monic irreducible cubics over $\mathbb{F}_3$	8	<code>trit</code> (3)	<code>ditrit</code> (9)	1
Frobenius orbits of $\text{GF}(27)$	11	<code>ditrit</code> (9)	<code>gf27Template</code> (27)	2
$\text{GF}(27)$	27	<code>gf27Template</code> (27)	<code>quadtrit</code> (81)	3

The index column is `clamp_census`. Three unrelated objects of three different sizes share the pair (`trit`, `ditrit`) (`metaNode_cubics_same_clamp`): the shelf is reused, and no security property is reproved. Nothing is tied to $\mathbb{F}_3$ or to small objects: over $\mathbb{F}_2$ the size index is the lattice’s capacity in bits (`clampIndex_two_eq_capacityBits`), and the Monster’s order is clamped between the binary templates of sizes 179 and 180 (`monster_clamp`).

5 Improvements

5.1 Soundness, quantitatively

Theorem 2.3 is a knowledge statement about *two* transcripts. What a verifier needs is a bound on *one* round against a *false* statement. The contrapositive supplies it exactly.

Definition 5.1 (`Gadget.goodChallenges`). For a statement x and a commitment a , put $\text{Good}(x, a) = \{ c \in \mathbb{F}_p : \exists z, Rz = a + cx \}$, the challenges a prover that has already sent a can still answer.

Theorem 5.2 (`Gadget.goodChallenges_subsingleton`, `Gadget.cheating_probability_le`). *Let p be prime and suppose x is false, i.e. there is no w with $Rw = x$. Then for every commitment a the set $\text{Good}(x, a)$ has at most one element, and hence*

$$\frac{|\text{Good}(x, a)|}{|\mathbb{F}_p|} \leq \frac{1}{p}.$$

For an honest prover, by contrast, $\text{Good}(x, Rr)$ is all of $\mathbb{F}_p$ (`Gadget.goodChallenges_of_isWitness`).

Proof (as formalised). If $c_1 \neq c_2$ both lay in $\text{Good}(x, a)$, the two accepting transcripts (a, c_1, z_1) and (a, c_2, z_2) share a commitment and differ in the challenge, so Theorem 2.3 produces w with $Rw = x$, contradicting falsity (`Gadget.challenge_unique_of_no_witness`). Hence $\text{Good}(x, a)$ is a subsingleton, so has cardinality at most 1, and division by $|\mathbb{F}_p| = p$ gives the bound. $\square$

The soundness error of one round is therefore $1/p$; n independent rounds give p^{-n} , and over $\mathbb{F}_3$ — the base this development works in — twenty rounds already put the cheating probability below $3 \cdot 10^{-10}$. Nothing here is conditional on a hardness assumption: it is a counting consequence of the extractor.

5.2 Composition: objects multiply, indices add

Theorem 5.3 (`clamp_prod`). *If X is clamped between $T(p^a)$ and $T(p^b)$, and Y between $T(p^c)$ and $T(p^d)$, then $X \times Y$ is clamped between $T(p^{a+c})$ and $T(p^{b+d})$.*

Theorem 5.4 (`clampIndex_mul_lower`, `clampIndex_mul_upper`). *For $p \geq 2$ and $m, n \geq 1$,*

$$\log_p m + \log_p n \leq \log_p(mn) \leq \log_p m + \log_p n + 1.$$

Together these say that a composite object never needs a gadget larger than the sum of its parts' gadgets plus a single witness coordinate, and never less than the sum. Composition of objects is addition of shelf indices, up to one trit.

5.3 The project itself as the object

Definition 5.5 (`ZKPoP.ProjectState`). The project's *state space* is the joint configuration of the five objects of Section 4:

$$\mathcal{P} = \text{Gal}(\text{GF}(27)/\mathbb{F}_3) \times \text{MetaNode} \times \text{IrreducibleCubics} \times \text{FrobOrbits} \times \text{GF}(27).$$

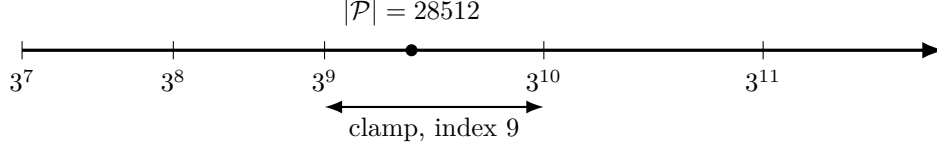
Theorem 5.6 (`card_projectState`). $|\mathcal{P}| = 3 \cdot 4 \cdot 8 \cdot 11 \cdot 27 = 28512$, *each factor being the state count proved elsewhere in the development.*

Theorem 5.7 (`project_clamp`, `project_clampIndex`, `project_no_encoding_lower`). $3^9 = 19683 \leq 28512 < 59049 = 3^{10}$, *so $\mathcal{P}$ is clamped between $T(3^9)$ and $T(3^{10})$, its tight index is 9 (unique, by Theorem 4.4), and there is no encoding of $\mathcal{P}$ into $T(3^9)$: the lower gadget is short by 8829 witnesses.*

Theorem 5.8 (`project_index_eq_parts_succ`). *The five component indices are 1, 1, 1, 2, 3, summing to 8, whereas `clampIndex` 3 $|\mathcal{P}| = 9$:*

$$\left(\sum_i \log_3 |X_i| \right) + 1 = \log_3 |\mathcal{P}|.$$

The whole costs exactly one trit more than the sum of its parts — the maximal slack permitted by Theorem 5.4, attained.



Theorem 5.9 (`The run`; `project_run_accepted`, `project_run_zero_knowledge`, `project_run_sound`, `project_cheating_probability_le`). *Fix the clamped presentation `projectClamped` : `Clamped` $\mathcal{P}$ $T(3^9)$ $T(3^{10})$ and let $x \in \mathcal{P}$ be a configuration with published statement $\sigma(x)$. Then:*

1. *every honest run proving possession of x is accepted;*
2. *for a fixed challenge, the runs are exactly the accepting transcripts, i.e. they are reproduced by the witness-free simulator: the verifier learns the clamp and nothing about which configuration was possessed;*
3. *two accepting transcripts on one commitment with different challenges extract a witness for $\sigma(x)$;*
4. *a prover with no witness for a false statement passes one round with probability at most $1/3$.*

So the “proof of possession of the project” is literal: a prover demonstrates possession of a complete configuration of the development’s objects — one Galois automorphism, one stage of the meme cycle, one irreducible cubic, one Frobenius orbit, one field element — in a single three-move run whose entire information content is the bracket $3^9 \leq |\mathcal{P}| < 3^{10}$.

6 What the verifier learns, and why it is an invariant

Corollary 6.1 (Noetherian reading). *The size index $\log_p |X|$ is conserved under every re-encoding of X : a re-encoding is a bijection, the bracket depends only on cardinality, and the tight clamp is unique (Theorem 4.4). It is therefore an invariant of the object and not of its presentation — the one quantity a run leaks, and the one quantity that survives every change of gadget.*

That is the sense in which the development’s slogan closes on itself. The cycle

$$\text{morphism} \longrightarrow \text{model} \longrightarrow \text{meme} \longrightarrow \text{medium} \longrightarrow \text{morphism}$$

is a symmetry of a four-element object; the protocol transports possession of a point of that object through a channel, and what is conserved along the way is a number attached to the object’s size. Symmetry above, conservation below: the analogy with Noether’s theorem is an analogy, not a theorem, and is stated here as such. What *is* a theorem is the conservation itself, in the precise sense of the corollary.

Scale of the artefact (not machine-checked). The development comprises 58 Lean modules, about 14,200 lines, and more than 850 theorem or lemma declarations; the ZKPoP layer is five of those modules. These counts are file statistics, obtained by counting source files and declaration headers, and are reported here for orientation only.

7 Related constructions

The three-move protocol for a linear preimage relation is classical, and the gadget of Section 2 is its matrix form; the extractor of Theorem 2.3 is the standard one. What is specific to this development is the organisation: capacity-indexed templates proved once and reused, encodings characterised exactly by a cardinality inequality, and clamps that are tight, unique, and identified with a lattice sandwich — together with the machine-checked guarantee that no step is missing.

A Formal names

All names live in the namespace `MetaMeme.ZKPoP`.

statement	Lean name	file
gadget, capacity	<code>Gadget</code> , <code>Gadget.capacity</code>	<code>Sigma.lean</code>
completeness	<code>Gadget.accepts_prove</code>	<code>Sigma.lean</code>
extractor, 2-special soundness	<code>Gadget.extract_spec</code>	<code>Sigma.lean</code>
simulator always accepted	<code>Gadget.accepts_simulate</code>	<code>Sigma.lean</code>
perfect HVZK	<code>Gadget.honest_eq_accepting</code>	<code>Sigma.lean</code>
templates, ladder	<code>template</code> , <code>ladder_three_capacities</code>	<code>Templates.lean</code>
GF(27) template	<code>gf27_equiv_witness</code>	<code>Templates.lean</code>
encoding iff fits	<code>nonempty_encoding_iff</code>	<code>Clamp.lean</code>
clamp iff bracket	<code>clamp_iff_clamped</code>	<code>Clamp.lean</code>
tight clamp, uniqueness	<code>tight_clamp</code> , <code>tightClamp_index_unique</code>	<code>Clamp.lean</code>
run on an encoded object	<code>Clamped.accepts_run</code> <i>et al.</i>	<code>Clamp.lean</code>
census of the five runs	<code>clamp_census</code>	<code>Runs.lean</code>
binary shelf, Monster	<code>monster_clamp</code>	<code>Runs.lean</code>
good challenges	<code>Gadget.goodChallenges</code>	<code>Project.lean</code>
soundness error $1/p$	<code>Gadget.cheating_probability_le</code>	<code>Project.lean</code>
composition of clamps	<code>clamp_prod</code>	<code>Project.lean</code>
index of a product	<code>clampIndex_mul_lower/upper</code>	<code>Project.lean</code>
project state space	<code>ProjectState</code> , <code>card_projectState</code>	<code>Project.lean</code>
its clamp and index	<code>project_clamp</code> , <code>project_clampIndex</code>	<code>Project.lean</code>
lower gadget too small	<code>project_no_encoding_lower</code>	<code>Project.lean</code>
whole vs. parts	<code>project_index_eq_parts_succ</code>	<code>Project.lean</code>
the run	<code>project_run_accepted/_zero_knowledge/_sound</code>	<code>Project.lean</code>

B Reproduction

The sources are the files `RequestProject/ZKPoP/{Sigma,Templates,Clamp,Runs,Project}.lean` of the accompanying project, with the module root `RequestProject/ZKPoP.lean`. Building the library checks every statement in this paper:

```
lake build RequestProject.ZKPoP
```

No file contains `sorry`, and the theorems depend only on `propext`, `Classical.choice` and `Quot.sound`, which can be confirmed with `#print axioms MetaMeme.ZKPoP.project_run_sound` and its analogues. This document is built from `paper/zkpop.tex`.